AI Usage Log

Sprint 2

Mohamad Alosta (40312377)

SOEN 341 - Software Process, Fall 2026

CareerConnect - Job Search and Application Tracking Platform

Contents

Prompt 3: US-05 Browse and View Job Offers

Prompt 4: US-14 Job Search Filters

Prompt 5: US-15 Apply for a Job, Task 15.1 and Fixes

Prompt 6: US-03 Clear Application Outcome Message

Prompt 7: Sprint 2 Documentation Review and Tracking

Prompt 3 - US-05 Browse and View Job Offers (Sprint 2)

- **Tool**: Claude
- **Date**: October 8 - 11, 2026
- **Purpose of AI Use**: Learning the project's tools, and step-by-step guidance and code review for US-05 (#10).
- **Chat Link or Prompt/Response**: No shareable link. My main requests are in the appendix below.
- **AI-Suggested Content**: The AI explained each concept using Michael's job postings as the example, split US-05 into steps with templates to fill in, and reviewed every change I made. Some code was provided directly by the AI (listed below).
- **Validation**: pnpm check (0 errors), pnpm test (31 passing, 4 new), a database check, and manual testing in the browser.
- **Decision**: Modified before use. I wrote most of the code from the templates, and the AI's parts are listed below.
- **Reflection**: I learned more about Svelte and TypeScript, and how my code connects to the database and the website.
- **Responsible Person**: Mohamad Alosta

Development and Understanding Process

The feature was built one small step at a time. For each step, the AI explained the concept first, I wrote the code, ran `pnpm check` / `pnpm test`, and the AI reviewed the result before moving on. I asked for explanations whenever something was unclear, for example:

- What database defaults are and why required columns added to an existing table need one.
- Why requirements and deadline should be required in the form validation rather than in the database (I chose this option after the AI explained both).
- Why the acceptance rate should be calculated rather than stored.
- How a form field's `name` connects to `data.get(...)` on the server.
- How to read TypeScript error messages (file, line, column, cause).
- Why `FormDataEntryValue` is a type and not a variable name.
- What the spread operator (`...posting`) does, and how to write the same thing field by field.
- How a date without a time is shifted by time zones (why `T00:00` is needed).
- The difference between `status` (can people apply?) and `published` (can job seekers see it?).

Work Done (by step)

Step	What was done	Who wrote it
1. Database columns (`schema.ts`)	Added `status`, `published`, `requirements`, `applicationDeadline`, `salaryMin`, `salaryMax`	Me, from the AI's explanation and a field table
Database migration	`alter table` SQL run in the Supabase SQL Editor	SQL provided by the AI; I ran it and verified the result
2. Validation (`job-posting.ts`)	Read and validated the new fields; requirements and deadline required; status open/closed; salary whole number >= 0 and min <= max	Me, from templates with blanks; `toSalary` helper and salary rule structure given as templates

Step	What was done	Who wrote it
2. Tests (`job-posting.test.ts`)	Updated the existing sample data and wrote 4 new tests	Me, with the AI explaining failures
3a. Form (`JobPostingForm.svelte`)	Requirements, deadline (date picker), status dropdown, published checkbox, salary min/max	Me, by copying existing fields as the AI explained; salary row structure provided by the AI
3b. Edit page (`edit/+page.server.ts`)	Convert `null` requirements/deadline to empty text for old postings	Field-by-field mapping provided by the AI; I placed and fixed it
3c. Job list (`jobs/+page.server.ts`, `+page.svelte`)	Hide unpublished jobs from job seekers; Open/Closed and Draft badges	Me, from templates with blanks
3c. Details access (`[id]/+page.server.ts`)	404 for unpublished jobs that aren't yours	Me, from a template with one blank
3d. Details page (`[id]/+page.svelte`)	Badges, requirements section, deadline, salary range, closed notice	Me, from templates; `formatDate` and `formatMoney` helpers provided by the AI

Errors and Bugs Found During Review

- `'application_deadline'` used as the form field name instead of `'applicationDeadline'` (would have made the deadline always "missing").
- `'true'` (text) used instead of `true` (boolean) as a default.
- Optional columns accidentally made required (`.notNull()`), which broke the create page.
- Test inputs that did not match the test's intent (valid values in a "required" test, explicit status in a "default" test), and a missing backtick.
- Status dropdown where both options were "selected" for new postings.
- Published checkbox hard-coded to `checked={true}` (would have published drafts on every edit).
- Draft badge checking `status` instead of `published`.
- Salary-only-maximum shown as "From" instead of "Up to".

Validation and Testing

- `pnpm check`: 0 errors at the end; `pnpm test`: 31 tests passing (4 new).
- Read-only database check after the migration: 6 new columns present with the right types and defaults; the existing posting stayed open and visible.
- Manual testing in the browser:
  - The browser blocks empty required fields; requirements made of spaces show my server error.
  - Minimum salary above maximum shows my error message.
  - A valid posting saves, and Edit shows all saved values.
  - Unticking Published hides the job from a job seeker's list.
  - Badges, requirements, deadline and salary display on the job pages.

Code Provided Directly by the AI

- SQL migration for the 6 new columns.
- `formatDate` and `formatMoney` helpers on the job details page.
- The field-by-field object returned by the edit page's `load` function.
- Structure of the salary row in the form and of the first two salary display cases.
- The local-only `vite.local.config.ts` (not committed) that moves Vite's cache outside Dropbox.

Other AI Use During This Period

- Reviewing my meeting minutes PR (#84) for formatting errors.
- Guidance on branch naming and cleaning up old branches.
- Setting up my development environment (installing Node.js and pnpm, creating `.env`, fixing a Vite cache error caused by Dropbox).
- Preparing for the demo (October 25 - 26): I asked the AI to explain the US-05 code chunk by chunk, split into the code I wrote, the code it wrote and the code we wrote together, and to walk me through the SQL I ran to add the new columns.

Appendix - Main Prompts

> *wait based on the project on git for the milestone we need US-05 not 6 now but can i do browse job offers or does it depend on job application to be done first*

> *ok so let us move through this one by one and if i understand it i write it down and u check it*

> *no dont give it to me yet i want to try and do it*

> *shouldnt requirements be required and also shouldnt we include acceptance rate*

> *let us go with A*

> *ok so i save what i have and let us start with step2*

> *Can u provide an explanation for US-3,5,14,15.1 for the codes u and i wrote separately and together so i can prepare for the demo*

Prompt 4 - US-14 Job Search Filters (Sprint 2)

- **Tool**: Claude
- **Date**: October 12 - 13, 2026, with one fix on October 25
- **Purpose of AI Use**: Step-by-step guidance and code review for US-14 (#19).
- **Chat Link or Prompt/Response**: No shareable link. My main requests are in the appendix below.
- **AI-Suggested Content**: The AI split US-14 into small steps with templates to fill in and reviewed each one. It also found and fixed a Clear search bug I noticed while testing.
- **Validation**: pnpm check (0 errors), pnpm test (4 new tests), manual searches in the browser, and acceptance tests #110 and #111.
- **Decision**: Modified before use. I wrote the search code from the templates, and the Clear search fix came from the AI.
- **Reflection**: I learned how a search form keeps the search in the URL, and how to build a query from optional filters.
- **Responsible Person**: Mohamad Alosta

Development and Understanding Process

I worked the same way as in US-05: one step at a time, I wrote the code, ran `pnpm check` and `pnpm test`, and the AI checked it before we moved on. Things I asked about:

- Why the search form uses `method="get"`, so the search is in the URL and survives a reload.
- What `ilike` does and why `%` goes around the text (upper and lower case don't matter, and part of a word is enough).
- Why the dropdown values are checked against the allowed choices before they reach the database.
- What the `mine` option ("Your postings") is for, and why it only filters for recruiters.
- Why a hidden field is needed in the form so "Your postings" stays on after searching.

Work Done (by step)

Step	What was done	Who wrote it
1. Reading the search (`job-search.ts`)	Trims the text fields, keeps only valid dropdown choices, cuts very long text, and gives empty values for missing fields	Me, from a template with blanks
1. Tests (`job-search.test.ts`)	4 tests for the cases above	Me
2. Filtering (`jobs/+page.server.ts`)	A condition for each filled-in field: keyword in the title or description, company, location, type and work mode	Me, from a template with blanks
3. Search form (`jobs/+page.svelte`)	The five search fields, a hidden field for "Your postings", and a "No jobs match your search" message with a Clear search link	Me, by copying the existing form fields as the AI explained
Fix (October 25)	Clear search now puts both dropdowns back to "Any"	Provided by the AI and explained after I reported the bug

Errors and Bugs Found

- While testing on October 25: after choosing Remote and clicking Clear search, the dropdown still showed Remote, so the next search filtered by Remote again. The chosen option was marked with `selected`, which the browser ignores once you've picked something yourself. Setting the value on the dropdown itself fixed it.
- Maher reviewed the PR and said the search works well, and that more filters could be added later.

Validation and Testing

- `pnpm check`: 0 errors. `pnpm test`: the 4 new search tests pass.
- Manual testing in the browser:
  - Part of a title in capitals finds the job.
  - Company and part of a city name find the right jobs.
  - Type and work mode work alone and together with the text fields.
  - A search with no matches shows the message and the Clear search link.
  - Clear search shows every job again with both dropdowns back to "Any".
- Acceptance tests #110 and #111 passed on October 25.

Code Provided Directly by the AI

- The Clear search fix (setting the value on each dropdown instead of on each option).

Other AI Use During This Period

- Renaming the test titles in every test file to the same "area: what should happen" style.
- Preparing for the demo (October 25 - 26): I asked the AI to explain the US-14 code chunk by chunk, split into the code I wrote, the code it wrote and the code we wrote together.

Appendix - Main Prompts

> *Can US-14 begin and can u navigate me through it?*

> *when we do clear search it doesnt revert from remote to any type*

> *US-14 tests check out*

> *Can u provide an explanation for US-3,5,14,15.1 for the codes u and i wrote separately and together so i can prepare for the demo*

Prompt 5 - US-15 Apply for a Job, Task 15.1 and Fixes (Sprint 2)

- **Tool**: Claude
- **Date**: October 17 - 25, 2026
- **Purpose of AI Use**: Guidance and code review for my part of US-15 (#20), task 15.1, and fixing problems found after Maher's PR #118.
- **Chat Link or Prompt/Response**: No shareable link. My main requests are in the appendix below.
- **AI-Suggested Content**: The AI gave me some templates for the application table, the validation and the apply rules, and reviewed my code. After PR #118, it found three bugs and set up the code for my deadline rule.
- **Validation**: pnpm check (0 errors), pnpm test, a database check for duplicate applications, and manual testing with a recruiter and a job seeker.
- **Decision**: Modified before use. I wrote the table, the validation and the apply rules with their tests, and the AI's fixes are listed below.
- **Reflection**: I learned how a unique rule in the database stops duplicate applications, and the difference between a function's parameters and the values passed to it.
- **Responsible Person**: Mohamad Alosta

Development and Understanding Process

US-15 was split between Maher and me. I built the application table first so the rest of US-15, and US-03 and US-20, could build on it. Things I asked about:

- Whether US-15 could start, and what it depended on (resumes and job postings already existed).
- How `unique(...).on(...)` makes one application per job per user, which is the point of the table.
- Why the tests use `{ ...openJob, status: 'closed' }`: `openJob` is a sample job, and the spread copies it and changes one field.
- Why a test file can't import the database schema file (it crashed the test run).
- Which time zone to use for "today": I asked about UTC or EST, and kept Eastern time (EST/EDT) so the deadline day ends at midnight in Montreal.
- What the page shows from that date, and adding the time to dates like "Applied Oct 22, 2026, 3:45 p.m. ET".

Work Done (by step)

Step	What was done	Who wrote it
Application table (`schema.ts`)	Links each application to a job, a user and a resume, with one application per job per user	Me, from a template
Statuses (`profile-options.ts`)	The five statuses, starting at Applied	Me, from a template
Validation (`application.ts`)	Checks that the chosen resume is a real id	Me, from a template
Apply rules (`applyProblem`)	Job not found, job closed, already applied	Me, from a template with blanks
Tests (`application.test.ts`)	6 tests for the validation and the rules	Me

Step	What was done	Who wrote it
Review of PR #118	Found the three problems listed above	The AI
Deadline rule (October 22 - 23)	No applying after the deadline, but still allowed on the deadline day	Me
Deadline tests	The day after the deadline is refused, the deadline day is fine	Me

Errors and Bugs Found During Review

- The unique rule was outside the table's list of rules and on the wrong columns.
- I passed values like `published: true` inside the function call instead of using the function's parameters.
- I imported the schema file in a test, which made the whole test file fail.
- In the deadline rule:
  - VS Code auto-imported an unrelated MySQL `date`, which I removed.
  - I first compared `alreadyApplied` (true or false) with a date.
  - Then I compared today with today, which is never later, before using the job's deadline.
- In the deadline tests, the expected message didn't match the rule's exact text, and I expected `true` where the rule returns `null` when nothing is wrong.

Validation and Testing

- `pnpm check`: 0 errors. `pnpm test`: all tests pass, including my 6 tests and the 2 deadline tests.
- Database check: the one-application-per-job rule is in place, and there are no duplicate applications.
- Manual testing with a recruiter and a job seeker:
  - "Application submitted successfully." stays on screen after applying.
  - Opening the job again, or in another tab, says "You have already applied to this job."
  - Cancel in the withdraw popup does nothing, and OK withdraws and shows a message.
  - A closed job, or one past its deadline, shows why you can't apply.
  - The recruiter sees the application and the resume.
- Acceptance test 15.1 passed on October 25.

Code Provided Directly by the AI

To save time, the AI wrote the parts below, which follow patterns already in the project. I read and tested each one.

- Passing the job's deadline and today's date to the apply rule, and a small helper that gives today's date in Eastern time.
- Keeping the success message on screen after applying, and showing messages when withdrawing.
- The time and "ET" on application dates and on the deadline.

Other AI Use During This Period

- Preparing for the demo (October 25 - 26): I asked the AI to explain my part of US-15 chunk by chunk, split into the code I wrote, the code it wrote and the code we wrote together.

Appendix - Main Prompts

> *can US15 start or does it depend on anything*

> *ok let us do US 15 my part and do whatever we can do further for 3 and 5*

> *so what we're doing is basically creating the uniqueness of applications*

> *since US-15 is done can u check it and see if it has any error*

> *u cant submit multiple applications to same role right?*

> *Can u provide an explanation for US-3,5,14,15.1 for the codes u and i wrote separately and together so i can prepare for the demo*

Prompt 6 - US-03 Clear Application Outcome Message (Sprint 2)

- **Tool**: Claude
- **Date**: October 17 - 25, 2026
- **Purpose of AI Use**: Guidance, templates and code review for US-03 (#8), plus the acceptance rate from US-05 (task 5.4).
- **Chat Link or Prompt/Response**: No shareable link. My main requests are in the appendix below.
- **AI-Suggested Content**: The AI planned US-03 around the existing features, gave me templates for the new rules, set up the parts that follow existing patterns, reviewed my code, and combined my page with Kimia's US-20 page.
- **Validation**: pnpm check (0 errors), pnpm test (101 passing), database checks, and manual testing with a recruiter and two job seekers.
- **Decision**: Modified before use. I wrote the rules and the database helpers, and the AI's code is listed below.
- **Reflection**: US-03 is what makes our project different, so I kept the rules strict: real reasons, no changes after sending, and a status that always matches the message.
- **Responsible Person**: Mohamad Alosta

Development and Understanding Process

US-03 had to wait for US-15, so I built the parts that didn't need applications first (the validation and the acceptance rate), then finished it once applications existed. Things I asked about or decided:

- Whether the message limit counts characters or words. I set the minimum to 100 characters because 30 was too short to explain a decision, and the maximum to 2000.
- Why next steps are required only for an offer, and cleared for a rejection.
- That `nextSteps` is a field of the form, not a function.
- How to count the days between two dates with `getTime()`, and why the order of the subtraction matters.
- Why the reminder uses `>`, so an application exactly 7 days old doesn't get a reminder yet.
- Whether a decision should be editable. I decided to keep it final, so the applicant never sees a message change after reading it, and to add a confirmation popup to catch mistakes instead.
- What a transaction is: saving the decision and the application's status together, so one is never saved without the other.
- That the acceptance rate belongs to US-05, even though it needs US-03's decisions.

Work Done (by step)

Step	What was done	Who wrote it
Validation (`application-outcome.ts`)	Offer or rejection, reasons between 100 and 2000 characters, next steps required for offers	Me, from templates
Validation tests	7 tests	Me
Database helpers (`outcomes.ts`)	Find a decision, check that the recruiter owns the application, count decisions for a job	Me, from templates
Acceptance rate (`acceptance-rate.ts`)	Offers out of all applications, nothing shown before the first decision	Me

Step	What was done	Who wrote it
Acceptance rate tests	4 tests	Me
Reminder rule (`reminders.ts`)	An application with no decision gets a reminder after 7 days	Me, from a template
Reminder tests	4 tests	3 set up by the AI, the 7-day one by me
No withdrawing after a decision (`withdrawProblem`)	One new rule	Me
Withdraw test	1 test	Filled in by the AI at my request
Status update (`saveOutcome`)	Sending a decision sets the application's status to Offered or Rejected	Me (the update line)
Database	The `application_outcome` table, and fixing the status of decisions sent while testing	SQL provided by the AI. I ran it in Supabase and checked the result.

Errors and Bugs Found During Review

- Validation: `<=` instead of `<` for the minimum, checking the reasons where I meant the next steps (a copy mistake), raising the limit to 10000 and then putting it back, and test messages that didn't exactly match the rules.
- Database helpers: joining the application's own id to the job's id instead of `jobPostingId`, filtering by the wrong id, and using `findIndex` where I needed `.length` to count.
- Acceptance rate: the division was flipped, and my test expected the wrong rounding (33 vs 34).
- Reminder rule: `getTime` without `()`, then subtracting in the wrong order, which gave negative days. My 7-day test was empty at first, so it passed without checking anything.
- Withdraw rule: I wrote `!decided`, which blocked every normal withdrawal. Four existing tests failed and showed the problem. The message also described the opposite case.
- Status update: I first used the table itself instead of the decision, then a `status` that only exists in browsers (it would have crashed on the server), then the function's own name, and then the word "Applied" instead of the application's id.

Validation and Testing

- `pnpm check`: 0 errors. `pnpm test`: 101 tests passing at the end.
- Database checks: the new table has the right columns and rules, and every decided application's status matches its decision (0 mismatches).

- Manual testing with a recruiter and two job seekers:
  - A reason under 100 characters is refused, and next steps are required for an offer.
  - The confirmation popup names the applicant, and Cancel sends nothing.
  - After sending, the form turns into the sent message, and it can't be sent twice.
  - The job seeker sees Offered or Rejected, the reasons and when it was sent, and can't withdraw anymore.
  - My Applications shows Offered or Rejected.
  - The acceptance rate goes from "not available yet" to 0% to 50%.
  - Another recruiter can't open the applications page.
- Acceptance tests 3.1 and 3.2.

Code Provided Directly by the AI

To save time, the AI wrote the parts below, which follow patterns already in the project. I read and tested each one.

- The `application_outcome` table in `schema.ts`.
- The decision form, the message the applicant sees, and the acceptance rate display.
- The decide action and the recruiter's page that shows each applicant's form or decision, and the decision on the job seeker's page.
- The confirmation popup, the "can't be changed" note and clearer error messages.
- The transaction around saving a decision, which my status update line runs inside.
- Combining my page with Kimia's US-20 status dropdown when both changed the same files.

Other AI Use During This Period

- Preparing for the demo (October 25 - 26): I asked the AI to explain the US-03 code chunk by chunk, split into the code I wrote, the code it wrote and the code we wrote together, and to walk me through the two SQL statements I ran (creating the decision table and fixing the status of old test decisions) line by line.

Appendix - Main Prompts

> *can we do US3 or does it depend on something*

> *is the message limit in characters or words and what abt min*

> *30 is too short*

> *nextsteps is a function?*

> *so u want me to do an if else statement ?*

> *are u sure we shouldnt make the decision outcome editable*

> *can u explain each line and walk me through them*

> *Can u provide an explanation for US-3,5,14,15.1 for the codes u and i wrote separately and together so i can prepare for the demo*

Prompt 7 - Sprint 2 Documentation Review and Tracking

- **Tool**: Claude
- **Date**: October 13 - 25, 2026
- **Purpose of AI Use**: Tracking Sprint 2 progress and checking the docs for errors, spelling and grammar.
- **Chat Link or Prompt/Response**: No shareable link. My main requests are in the appendix below.
- **AI-Suggested Content**: The AI compared the Sprint 2 guidelines with our repository and GitHub, pointed out what was missing or out of date, and suggested using the Sprint 1 docs as a base for Sprint 2's, which I reviewed, edited and wrote from its suggestions (see the "Work Done" table below). It also counted each member's contributions and proofread the docs.
- **Validation**: I checked everything against GitHub and the guidelines, and added what the AI couldn't know, like Andrew dropping the course, US-18 moving to Sprint 3 and my hours.
- **Decision**: Modified before use.
- **Reflection**: Docs need regular checks against GitHub. Small mistakes add up, and an early merge can leave fixes behind.
- **Responsible Person**: Mohamad Alosta

Work Done

Document	What was done	Who did it
Sprint 2 plan (`sprint-2.md`)	Scope, capacity, story points, velocity, risks and work plan, kept up to date as stories finished	The AI suggested making something similar to sprint-1.md, so I edited it and created a version for Sprint 2.
Task breakdown	Sprint 2 section, with each story's tasks copied exactly from its GitHub issue	The AI suggested adding Sprint 2 to the same breakdown as Sprint 1.
Contribution log	Commits, pull requests, reviews and issues for each member, and the meetings	The AI counted from GitHub, and I reviewed them and filled in my own hours
Meeting minutes	Reviewed my October 22 minutes	I wrote them. The AI found the mistakes listed below.
README	What's implemented so far, and the Sprint 2 scope	The AI suggested adding Sprint 2 related links.
Summary report	One-page Sprint 2 report, exported to PDF	The AI suggested example docs, and I edited the file to represent our report.

Errors Found by AI and Fixed by Me

- "Micheal" instead of "Michael" in all four meeting minutes.
- The October 22 minutes pointed to issue #94 instead of #120, had "the the", and didn't say what was discussed (progress on US-20).
- "corefeatures" in the report and "unkown" in a test title.
- Statuses that were out of date as stories finished (US-05, US-14, US-15, US-20, US-03), and US-18 still listed as a Sprint 2 story.
- Contribution counts that changed after new pull requests and reviews.

- Acceptance tests whose criteria were reworded instead of copied exactly from the user story.
- Tables padded with spaces by VS Code's Format Document, which made every line look changed.

Validation

- Compared the docs with GitHub after every change (issues, pull requests, merges and reviews).
- Checked that the Sprint 2 docs match each other: same statuses, same PR numbers, same velocity.
- Opened the report PDF to check that it fits on one page and the formatting is right.

Appendix - Main Prompts

> *which docs need to be added/changed?*

> *can u check the numbers of commits and merged PRs etc*

> *Andrew dropped the course, so what are necessary changes document wise?*

> *can u check what requirements are missing for sprint 2 and which are only here*